CHAPTER 1

NON-FUNCTIONAL SYSTEM CHARACTERISTICS

1 Availability 2
2 Reliability 2
3 Scalability 4
4 Maintainability 10
5 Fault Tolerance 11
6 Non-Functional Requirements for System Design Interviews 14
7 Let AI Evaluate Your Understanding of Non-Functional Requirements . . . 21

Review key non-functional characteristics like availability, reliability, scalability, and fault tolerance in System Design.

1 Availability

1.1 What is availability?

Availability is the percentage of time that some service or infrastructure is accessible to clients and is operated upon under normal conditions. For example, if a service has 100% availability, it means that the said service functions and responds as intended (operates normally) all the time.

Measuring availability

Mathematically, availability, `A`, is a ratio. The higher the `A` value, the better. We can also write this up as a formula:

$$\text{A (in percent)} = \frac{\text{(Total Time - Amount Of Time Service Was Down)}}{\text{Total Time}} \times 100$$

We measure availability as a number of nines. The following table shows how much downtime is permitted when we're using a given number of nines.
The Nines of Availability

Availability Percentages versus Service Downtime			
Availability %	**Downtime per Year**	**Downtime per Month**	**Downtime per Week**
90% (1 nine)	36.5 days	72 hours	16.8 hours
99% (2 nines)	3.65 days	7.20 hours	1.68 hours
99.5% (2 nines)	1.83 days	3.60 hours	50.4 minutes
99.9% (3 nines)	8.76 hours	43.8 minutes	10.1 minutes
99.99% (4 nines)	52.56 minutes	4.32 minutes	1.01 minutes
99.999% (5 nines)	5.26 minutes	25.9 seconds	6.05 seconds
99.9999% (6 nines)	31.5 seconds	2.59 seconds	0.605 seconds
99.99999% (7 nines)	3.15 seconds	0.259 seconds	0.0605 seconds

Table 1.1: The Nines of Availability

Availability and service providers

Each service provider may start measuring availability at different points in time. Some cloud providers start measuring it when they first offer the service, while some measure it for specific clients when they start using the service. Some providers might not reduce their reported availability numbers if their service was not down for all the clients. The planned downtimes are excluded. Downtime due to cyberattacks might not be incorporated into the calculation of availability. Therefore, we should carefully understand how a specific provider calculates their availability numbers.

2 Reliability

2.1 What is reliability?

Reliability is the probability that the service will perform its functions for a specified time. It measures how the service performs under varying operating conditions.
We often use **mean time between failures (MTBF)** and **mean time to repair (MTTR)** as metrics to measure reliability.

$$\text{MTBF} = \frac{\text{Total Elapsed Time - Sum of Downtime}}{\text{Total Number of Failures}}$$

$$\text{MTTR} = \frac{\text{Total Maintenance Time}}{\text{Total Number of Repairs}}$$

(We strive for a higher MTBF value and a lower MTTR value.)
Reliability and availability are often confused, but they measure different aspects of system performance. While reliability focuses on how consistently a service operates without failure, availability considers how often it is accessible when needed. Understanding both is crucial because even a highly reliable system can have low availability if downtime or repairs take too long. Let 's explore how these two concepts are interconnected:

Reliability and availability

Reliability and availability are two important metrics to measure compliance of service to agreed-upon service level objectives (SLO)A service-level objective (SLOs) is a key element of a service-level agreement (SLA) between a service provider and a customer. SLOs are agreed upon as a means of measuring the performance of the Service Provider and are outlined as a way of avoiding disputes between the two parties based on a misunderstanding..
The measurement of availability is driven by time loss, whereas the frequency and impact of failures drive the measure of reliability. Availability and reliability are essential because they enable the stakeholders to assess the health of the service.
Reliability (`R`) and availability (`A`) are two distinct concepts, but they are related. Mathematically , `A` is a function of `R`. This means that the value of `R` can change independently, and the value of `A` depends on `R`. Therefore, it's possible to have situations where we have:

- low `A`, low `R`
- low `A`, high `R`
- high `A`, low `R`
- high `A`, high `R` (desirable)

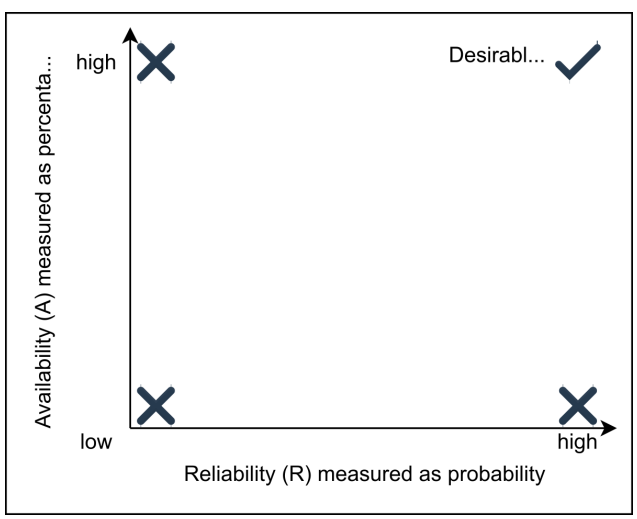


Figure 1.1: Availability as a function of reliability

Note: There are many variations of the MTBF metric, such as mean time to failure (MTTF). Usually, we use MTTF instead of MTBF for those cases where a failed component is replaced due to irreparable problems. A bad disk or a failed bulb are examples of irreparable faults where a replacement is required.

Quiz:
Question 1: What is the difference between reliability and availability?
Answer: Reliability measures how well a system performs its intended operations (functional requirements). We use averages for that (Mean

Time to Failure, Mean Time to Repair, etc.) Availability measures the percentage of time a system accepts requests and responds to clients. Example 1: A certain system may be 90% available but only reliable 80% of the time. Example 2: Suppose we consider our "system" the stuff inside a data center (hardware + software). Let's assume this data center suffers a network failure such that no outsider traffic is coming in and no insider traffic is going out. In this case, instantaneous availability might be zero (because clients cannot reach the service) even though inside the data center, all systems are perfectly functioning (instantaneous reliability 100%). We use both of them (reliability and availability) in different contexts. For example, storage vendors often quote MTTF for their disks. Most online services use uptime (as a measure of availability) in their SLAs. For example, the uptime of EC2 virtual machines is 99.95%.

3 Scalability

3.1 What is scalability?

Scalability refers to a system's ability to handle an increasing workload or a growing number of users without compromising performance.
A search engine, for example, must accommodate increasing numbers of users as well as the amount of data it indexes. In simple terms, a scalable system can grow to meet demand while maintaining responsiveness and reliability as more users, data, or features are added.
Example: In its early days, as Twitter's popularity grew, its system frequently crashed and displayed the infamous fail whale. The problem was not poor code quality but poor scalability. The system was unable to keep up with the surge of users and real-time activity.
Scalability ensures that such spikes in traffic are handled efficiently and effectively. Without it, systems slow down, fail, or frustrate users, causing them to leave for smoother experiences elsewhere.

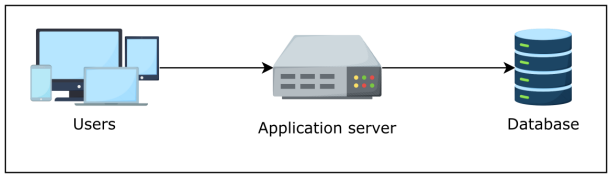


Figure 1.2: A simple system handling users' requests

The workload of a system can vary by type:

- **Request workload:** The number of requests served by the system.
- **Data or storage workload:** The amount of data stored, processed, or retrieved by the system.

3.2 Dimensions of scalability

Scalability can be viewed along different dimensions:

- **Size scalability:** The ability to add users or resources easily without redesigning the system.
- **Administrative scalability:** The capacity for a growing number of users or organizations to share the same distributed system efficiently.
- **Geographical scalability:** The ability of the system to maintain acceptable performance across regions as it expands geographically.

3.3 Why scalability matters

Scalability goes beyond simple growth. It represents the system's ability to remain resilient and adaptable in response to changing demands. Without proper scalability, systems may experience downtime, high latency, or reduced performance during periods of high activity.

When do we need to scale?

A system should be scalable considering these factors:

- **Anticipating future growth:** Systems should anticipate user and data growth and scale accordingly.
- **Boosting system performance:** Distributing the workload improves response times and user experience.
- **Ensuring availability:** Scaling ensures uptime during high-traffic events.
- **Expansion to new regions:** Scaling supports growth as services go global.
- **Feature expansion:** Adding new functionality often demands more resources.
- **Integrating third-party services:** Systems should be able to handle the additional load from APIs or payment gateways.

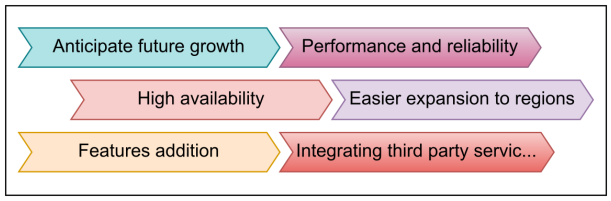


Figure 1.3: An important factor to consider to scale a system

Overall, scaling a system is essential in System Design, as it ensures that the system remains available and can grow over time to meet the business requirements.

Quiz:
Question 1: What happens if a system isn't scaled properly?
Answer: If we don't scale a system, it will struggle to handle increased users,

performance demands, and new features leading to slowdown, crashes, and poor user experience. This can result in service outages, customer dissatisfaction, and potential revenue loss.

3.4 Different approaches to scalability

Here are the primary methods for implementing scalability in a system.

Vertical scalability (scaling up)

Vertical scaling, also known as scaling up, refers to upgrading the hardware resources of an existing system to handle an increasing load.
It may include adding more CPU cores, RAM, or storage capacity to boost performance. Vertical scaling allows teams to expand their current hardware or software capacity, but only up to the limits of a single server. It is simpler to implement and manage compared to other approaches and can significantly improve performance for systems running on a single node.
However, it also comes with some drawbacks. The scalability is limited by the maximum capacity of a single machine, and upgrades may cause temporary downtime. Moreover , high-performance components tend to be expensive, which increases overall costs.

> **Note:** This approach works best for applications with predictable workloads that need an immediate performance boost without requiring major architectural changes.

Horizontal scalability (scaling out)

Horizontal scaling, also known as scaling out, involves adding more machines to the network to distribute the workload across multiple servers.
Instead of upgrading a single system, additional servers are deployed to share the processing load. This method offers higher scalability, better fault tolerance, and improved reliability, as failures in one machine do not impact the rest of the system.
It is also more cost-efficient because it can use commodity hardware instead of specialized high-end machines.
However, horizontal scaling introduces added complexity, as managing and maintaining a distributed system is more difficult. Communication between servers may also increase latency; therefore, the application architecture must support distributed operations to ensure effective scaling.

> Horizontal scaling is particularly suitable for systems that expect rapid growth or fluctuating workloads, requiring flexibility and elasticity.

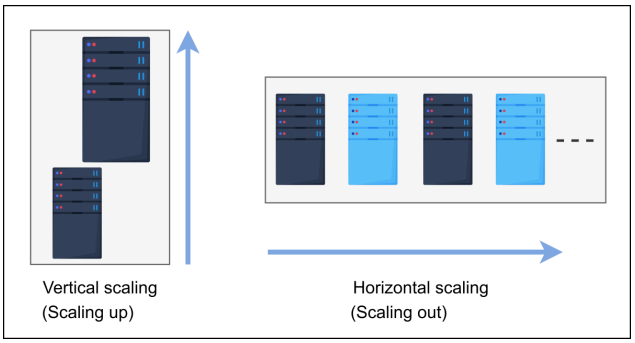


Figure 1.4: Vertical scaling versus horizontal scaling

Instead of relying solely on vertical or horizontal scaling, many modern systems use autoscaling.
Autoscaling automatically adjusts the number of resources allocated to a system in response to real-time demand. It continuously monitors metrics such as CPU usage, network traffic, or queue length and dynamically adds or removes resources to maintain optimal performance.
This approach enables systems to remain responsive during sudden surges in traffic while conserving resources during periods of low activity.

3.5 Scalability techniques

Beyond adding hardware, specific architectural techniques are fundamental to building a scalable system.

- **Load balancing:** It ensures that user traffic is evenly distributed among the available servers, preventing any single server from being overloaded. In this way, the system can handle increased load as no single server is overwhelmed and fails.

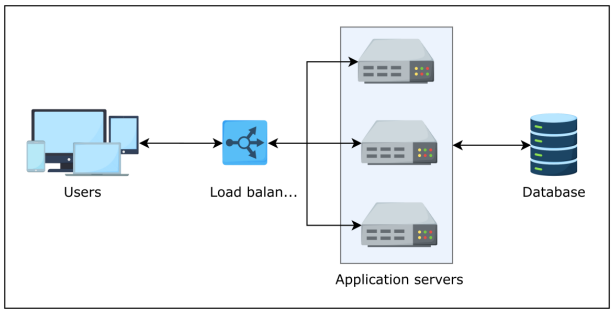


Figure 1.5: Load balancing between available servers

- **Caching and Content Delivery Networks (CDNs):** Caching stores commonly accessed data in temporary storage (like memory or a dedicated caching server), reducing the need for repeated requests to the database. Additionally, distributing static content, such as videos and images, through CDNs close to users further reduces the system's load.

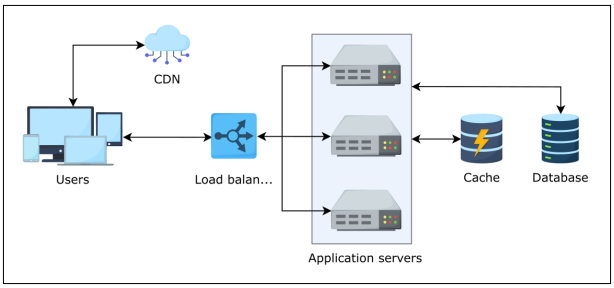


Figure 1.6: Caching frequently accessed data in cache and CDN

- **Data replication and sharding:** Data replication duplicates data across multiple servers to enhance scalability and fault tolerance, while sharding divides data into smaller partitions across databases to improve performance and scalability.

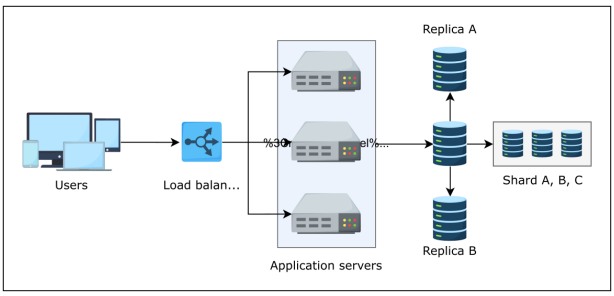


Figure 1.7: Data replication and sharding

- **Microservices architecture:** A microservice architecture improves scalability by breaking down applications into smaller, independent services that can be handled or managed independently. It also allows each service to scale independently according to demand, without affecting the performance of other services.

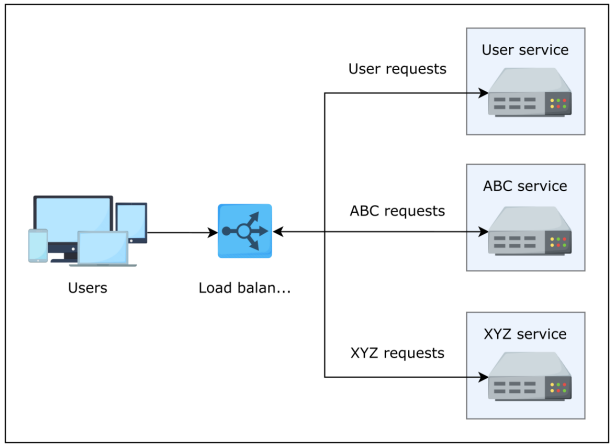


Figure 1.8: Microservices architecture

Note: By understanding and implementing the right scalability techniques effectively, engineers can build systems that are future-proof, survive the ever-increasing demand, and are available to meet upcoming challenges.

Quiz:
Question 1: How do you think companies like Netflix or Amazon handle millions of users at once?
Answer: They use key techniques, such as caching, load balancing, sharding, and replication, to distribute workloads and maintain fast, reliable services for everyone.

3.6 Best practices for System Design scalability

Following best practices can help make a system more resilient and scalable to ever-increasing demand:

- **Mitigating performance bottlenecks:** Analyze and determine the performance bottlenecks in the system. These can be caused by factors such as inefficient database queries and algorithms. Identifying and mitigating such bottlenecks is essential to ensure scalability.

- **Efficient resource utilization:** Available resources should be utilized efficiently. Implement a queuing mechanism to process incoming requests and use worker servers to handle background tasks independently. Effective caching can also help improve system efficiency.

  Quiz:
  Question 1: What are worker servers, and what types of tasks can be assigned to these workers?
  Answer: The worker servers handle time-consuming tasks that don't require immediate user feedback, such as processing images or running data analysis jobs. We can ensure that the platform's core functions remain fast and efficient.

- **Minimizing network latency:** Network latency can significantly impact a system's performance. Reducing latency through minimizing network hops, utilizing caching, and optimizing data transfer can help scale a system.

- **Improving data storage and access:** How data is stored and accessed also impacts scalability. Scalable and distributed storage, caching, efficient data access patterns, indexing, data replication, and partitioning can help improve scalability.

- **Choosing the right and modern technologies:** Selecting the right technologies is a significant advantage for scalability. For example, selecting the most efficient algorithm, optimizing database queries, and choosing SSD over HDD can have a significant impact on the system.

Quiz:
Question 1: How does cloud computing make scalability easier?
Answer: With cloud computing, scaling becomes more flexible, allowing you to add or reduce resources on demand. It makes handling traffic spikes and growth much simpler without the need for physical hardware upgrades.
Question 2: What role does DevOps play in maintaining a scalable system?
Answer: DevOps practices streamline scaling by automating infrastructure management, enabling faster deployments, and ensuring that your system can grow and adapt efficiently.

3.7 Scalability challenges and trade-offs

Along with the techniques and best practices for scaling a system, engineers face several challenges:

- **Cost:** To scale a system, we often need to add more resources, which can directly increase its cost. This can be a direct trade-off to achieve scalability.
- **Consistency:** Maintaining data consistency becomes increasingly difficult across distributed systems as they scale.
- **Security:** Ensuring consistent security policies across services and data becomes challenging as the system grows.
- **Complexity:** Systems become more complex to manage and troubleshoot as we scale.

  Quiz: Question 1: How can we efficiently distribute and balance traffic across multiple servers to handle increasing loads? **Answer:** Implementing load balancers to distribute incoming requests evenly across a server pool can help efficiently handle the increasing load.

3.8 Scalability in action (real-world examples)

The following are some examples of scalable real-world systems:

- **Google Search:** Built on a massively scalable architecture that processes billions of queries daily.
- **Netflix:** Utilizes cloud-based infrastructure (Amazon Web Services) to handle millions of concurrent requests and deliver seamless streaming to users.
- **Facebook:** A highly scalable system that handles requests and data from millions of users across the globe.
- **Uber:** Designed to handle millions of ride requests globally.

3.9 Conclusion

Scalability in System Design is a vital element determining how well a system can grow, adapt, and perform under increasing demands.
Whether scaling to accommodate more users, expanding to new regions, or introducing new features, maintaining performance and reliability is key to ensuring a seamless user experience. By implementing best practices such as load balancing, sharding, and data replication, and understanding the associated trade-offs, businesses can ensure their systems remain resilient and responsive in the face of growth.
As we look to the future, AI and machine learning are becoming more prominent in scaling systems intelligently. These technologies can automate resource management, predict traffic patterns, and optimize infrastructure in ways that were previously unimaginable.

4 Maintainability

4.1 What is maintainability?

Besides building a system, one of the main tasks afterward is keeping the system up and running by finding and fixing bugs, adding new functionalities, keeping the system's platform updated, and ensuring smooth system operations. One of the salient features to define such requirements of an exemplary system design is **maintainability**. We can further divide the concept of maintainability into three underlying aspects:

1. **Operability**: This is the ease with which we can ensure the system's smooth operational running under normal circumstances and achieve normal conditions under a fault.

2. **Lucidity**: This refers to the simplicity of the code. The simpler the code base, the easier it is to understand and maintain it, and vice versa.

3. **Modifiability**: This is the capability of the system to integrate modified, new, and unforeseen features without any hassle.

Measuring maintainability

Maintainability, `M`, is the probability that the service will restore its functions within a specified time of fault occurrence. `M` measures how conveniently and swiftly the service regains its normal operating conditions. For example, suppose a component has a defined maintainability value of 95% for half an hour. In that case, the probability of restoring the component to its fully active form in half an hour is 0.95.

> **Note:** Maintainability gives us insight into the system's capability to undergo repairs and modifications while it's operational.

We use (mean time to repair) MTTR as the metric to measure `M`.

$$\text{MTTR} = \frac{\text{Total Maintenance Time}}{\text{Total Number of Repairs}}$$

In other words, MTTR is the average amount of time required to repair and restore a failed component. Our goal is to have as low a value of MTTR as possible.

Maintainability and reliability

Maintainability can be defined more clearly in close relation to reliability. The only difference between them is the variable of interest. Maintainability refers to `time-to-repair`, whereas reliability refers to both `time-to-repair` and the `time-to-failure`. Combining maintainability and reliability analysis can help us achieve availability, downtime, and uptime insights.

5 Fault Tolerance

5.1 What is fault tolerance?

In the real world, large-scale applications utilize hundreds of servers and databases to accommodate the requests of billions of users, and store vast amounts of data. These applications require a mechanism that ensures data safety and avoids the recalculation of computationally intensive tasks by eliminating a single point of failure.
Fault tolerance refers to a system's ability to execute persistently even if one or more of its components fail. Here, components can be software or hardware. Conceiving a system that is 100 percent fault-tolerant is practically very difficult.

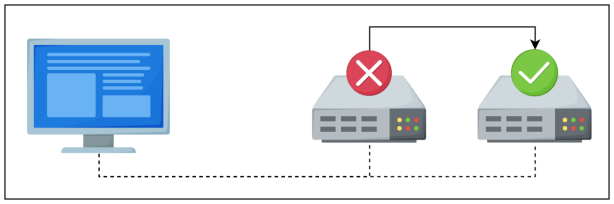


Figure 1.9: Fault tolerance in action: when one server fails, another seamlessly takes over

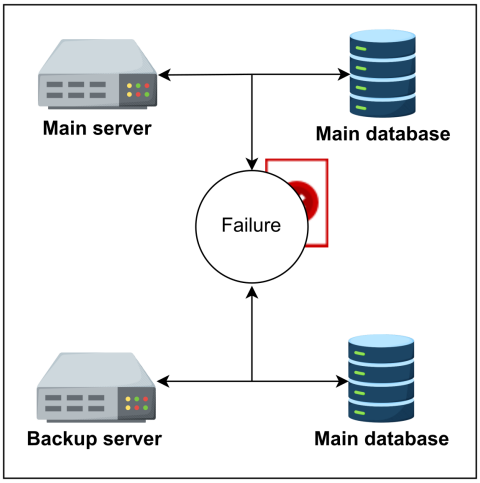
Two key qualities make fault tolerance essential: availability and reliability.
Availability ensures that the system remains accessible and can receive client requests at any time. Reliability , on the other hand, ensures that the system consistently processes those requests and performs the correct actions every time.
Fault-tolerant systems aim to ensure high availability of the system by preventing disruptions arising from a *single point of failure*.
There are two fault-tolerant approaches:

- **Fault-removal:** This can be either forward error recovery or backward error recovery.
- **Fault-masking:** when the presence of one defect hides the presence of another defect in the system.

For fault tolerance with zero downtime (constantly active), a hot failover must be implemented, which instantly transfers workloads to a functioning backup system.
If maintaining a constantly active standby system is not required, a warm or cold failover can be used, where the backup system loads and starts the workloads only when needed. A warm or cold failover is slower because the system must load and initialize resources before becoming active.
Fault-tolerant computing also provides limited protection against software failures, which remain a major cause of downtime and data center outages for many organizations.
With that understanding, let's now explore some common techniques used to achieve fault tolerance.

5.2 Advantages of fault-tolerant systems

The primary purpose of creating fault tolerance is to prevent (or at least minimize) a situation where the system's functionality becomes unavailable due to a fault in one or more of its components.
Fault tolerance is necessary for systems that are used to protect people's safety (such as air traffic control hardware and software systems) and in systems where security, data protection, data integrity, and high-value transactions all depend on.
Fault-tolerant systems provide an excellent safeguard against equipment failure, but they can be extraordinarily expensive to implement because they require a fully redundant set of hardware that needs to be linked to the primary system.



5.3 Fault tolerance techniques

Failure occurs at the hardware or software level, which eventually affects the data. Fault tolerance can be achieved through various approaches, considering the system's structure and design. Let 's discuss the techniques that are significant and suitable for most designs.

Forward and Backward error recovery

Forward error recovery involves identifying the error and, based on this knowledge, correcting the system state containing the error. Exception handling in high-level languages like *Ada* and *PL/1* provides a system structure that supports forward recovery.
Backward error recovery corrects the system state by restoring the system to its stable state that existed prior to the fault's manifestation.

Replication One of the most widely used techniques is *replication-based fault tolerance*.
With this technique, we can replicate both the services and the data. We can swap out failed nodes with healthy ones and a failed data store with its replica. A large service can make the switch transparently without impacting end customers.
We create multiple copies of our data in separate storage.
All copies must be updated regularly to maintain consistency when any updates occur in the data. Updating data in replicas is a challenging job. When a system requires strong consistency, we can update data in replicas synchronously.
However, this reduces the system's availability.
We can also asynchronously update data in replicas when we can tolerate eventual consistency, resulting in stale reads until all replicas converge. Thus , there is a trade-off between the two consistency approaches. We often compromise on either availability or consistency in the event of a network partition, a reality outlined in the CAP theorem.

Checkpointing **Checkpointing** is a technique that saves the system's state in stable storage for later retrieval in case of failures due to errors or service disruptions. Checkpointing is a fault-tolerance technique performed in multiple stages at regular time intervals.
When a distributed system fails, we can retrieve the last computed data from the previous checkpoint and resume work from that point.
Checkpointing is performed for individual processes in a system in a way that represents the global state of the system's actual execution. Depending on the state, we can divide checkpointing into two types:

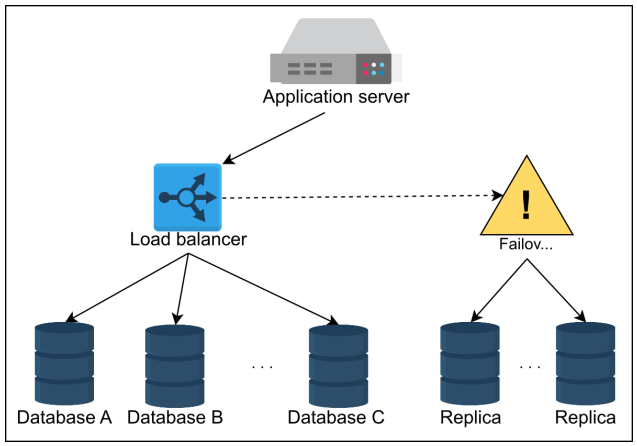


Figure 1.10: Replication-based fault tolerance

- **Consistent state:** A state is consistent in which all the individual processes of a system have a consistent view of the shared state or sequence of events that have occurred in a system. Snapshots taken in consistent states have data in coherent states, representing a possible situation of the system. For a checkpoint to be consistent, typically, the following criteria are met:
  - All updates to data that were completed before the checkpoint are saved. Any updates to data that were in progress are rolled back as if they hadn't been initiated.
  - Checkpoints include all the messages that have been sent or received up until the checkpoint. No messages are in transit (in-flight) to avoid cases of missing messages.
  - The relationships and dependencies between system components and their states align with what would be expected during normal operation.
- **Inconsistent state:** This is a state where discrepancies exist in the saved state of different processes within a system. In other words, the checkpoints across different processes are not coherent and coordinated.

Let's look at an example to better understand consistent and inconsistent states. Consider three processes represented by i, j, and k. Two messages, m_1 and m_2, are exchanged between the processes. Other than that, we have one snapshot/checkpoint saved for each process represented by $C_{1,i}$, $C_{1,j}$, and $C_{1,k}$, where 1 represents the number of snapshots for a process and the lowercase letter represents the process itself.

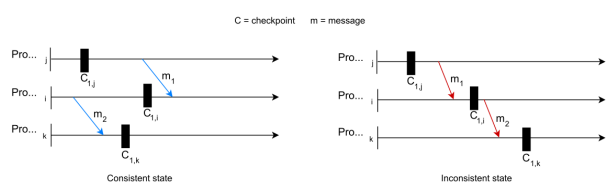


Figure 1.11: Checkpointing in a consistent and inconsistent state.

In the illustration on the left, the first checkpoints at processes j and i are consistent because m_1 is sent and received after the checkpoints. On the contrary, in the right-hand illustration, the first checkpoint at process j doesn't know about m_1, while the first checkpoint at process i recorded the reception of message m_1. Therefore, it's an inconsistent state.
The left-hand illustration represents a consistent state, also because no communication is being performed among the processes when the system performs checkpointing. On the right side, we can see that the processes communicate through messages when the system performs checkpointing.

6 Non-Functional Requirements for System Design Interviews

Designing a system to meet its nonfunctional requirements (NFRs) presents a significant challenge. This involves managing critical trade-offs between competing NFRs such as scalability, availability, performance, and security.

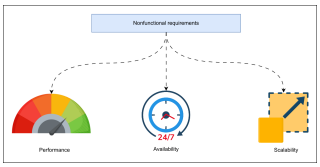


Figure 1.12: Nonfunctional requirements discussed in this article

Take this key System Design interview question, for example:

- How can you design a scalable and performant e-commerce website that can handle millions of requests per second?

While the engineer will be able to design the system to meet all functional requirements, making the design scalable and still achieving low latency on requests will remain a challenge.
Today, through this lesson, we'll discuss a few essential strategies for meeting NFRs in our designs. These strategies will prepare us to confidently navigate System Design interviews at top tech companies.
Let's get started!
Disclaimer: In the previous lessons, we defined certain NFRs. Please note that identifying and achieving an NFR are two different things. In this lesson, our focus will be on achieving non-functional requirements.

6.1 Common non-functional requirements

Let's discuss common non-functional requirements that interviewers focus on and learn how to meet them effectively when tackling System Design interview questions. The common non-functional requirements that we will address in this lesson are:

1. Performance
2. Availability
3. Scalability

1) Performance

Performance is a known NFR that determines the system's ability to respond to user requests and process data efficiently.
For example, when designing a messaging service, the interviewer might ask questions like: How to deliver messages with low latencyLow latency refers to minimal time delays? To achieve low latency, candidates must select an efficient two-way communication protocol. For this, they can opt for WebSocket. This is a single example of achieving performance. We will see more examples in the upcoming approaches section.
Let's examine various approaches to achieving optimal performance.

Approaches to achieve performance **Caching:** Implementing an effective cache mechanism is one of the methods to achieve optimal performance. It stores frequently accessed data, reducing the need for repeated computations and minimizing user-perceived latency.

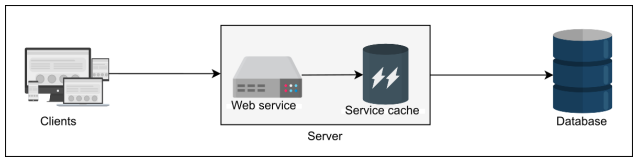


Figure 1.13: Web service uses service cache to access frequently accessed data to ensure low latency

Let's assume an X (formerly Twitter)-like system where a service is dedicated to generating the timelineThe timeline includes a stream of posts and various recommendations based on the user's interests and followers' activity (such as their posts, reposts, likes, etc.)..
Let's call it the timeline service. Now , the question that interviewers commonly ask related to the timeline is: Does the timeline service generate a timeline of each celebrity's followers when a celebrity posts something? As celebrities have millions of followers, it impacts the system's performance to create timelines for all of them.
To address this question, we need to analyze the followers first.
Not every follower uses X all the time. As a first step, we can divide those followers into active usersActive users are those who frequently use their X accounts. and inactive usersInactive users are those who used their accounts a long time ago (say, more than three months).. For inactive users, the timeline service will not generate the timeline instantly. For active users, we will introduce a cache.
Let's call it the feed cacheIt is a distributed cache like Redis for storing the timeline of active users..
This cache prepopulates the timeline for active users. When active users request a timeline, the timeline service immediately retrieves the timeline from the feed cache and appends the celebrity post, returning it to the client with minimal latency.

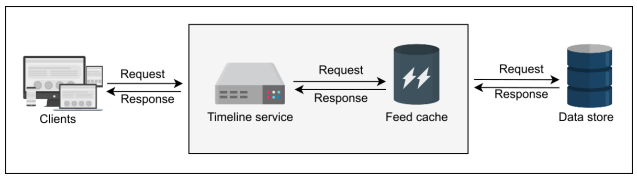
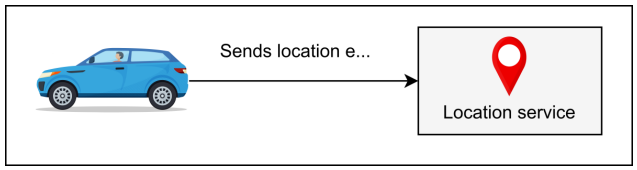


Figure 1.14: Fetching a timeline for active users from the feed cache

Additionally, a cache mechanism is usually implemented in each system layer to ensure decoupling and low latency.
Algorithm/Data Structure selection: Choosing efficient algorithms and data structures is another approach to enhancing the system's performance.
An efficient algorithm or data structure minimizes processing time and improves overall system performance. For example, a candidate may be asked which data structure would be suitable for efficiently and frequently (every four seconds) storing a driver's position in the ride-hailing system.



One possible solution is the Quadtree data structure, which is a suitable option for ensuring optimal performance in this case.
It receives data from the driver every 4 seconds, and the driver relocates within the Quadtree based on the new location. However, updating a Quadtree at such a high frequency introduces computational overhead,

which can increase latency.
Candidates should consider whether a Quadtree is optimal in this scenario and may explore combining different approaches to achieve better performance and scalability.
Load balancing: Distributing incoming traffic evenly among different servers (load balancing) is another strategy to achieve high performance. For example, with millions of users on an e-commerce website, multiple requests can arrive in a matter of seconds. Load balancing is useful in such situations to distribute the load on a server, allowing it to handle only the load it can manage. Load balancers distribute user requests across multiple servers to prevent bottlenecks and ensure optimal server performance.

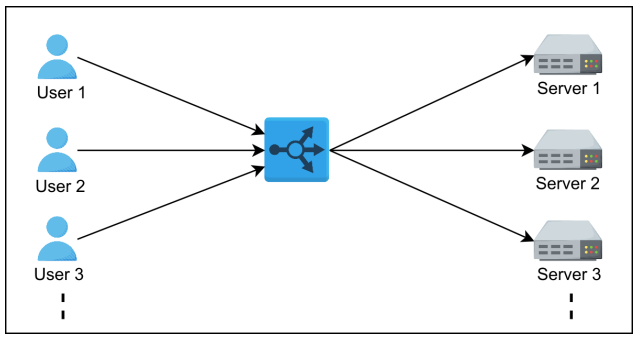


Figure 1.15: Distributing user requests across multiple web servers

2) Availability

The system's availability is another non-functional requirement that describes how effectively it maintains users' accessibility and uptime. Generally , a system with 99.999% uptime is considered good availability. This percentage of availability is equivalent to less than 6 minutes of downtime per year. Achieving this amount of availability is very challenging. However, providing 99.999% availability helps retain a large number of users.
For example, when designing an online shopping website, availability is crucial since customers often use the site to browse products, make purchases, and track their orders. Any downtime might lead to lower sales and disappointed customers.
Let's discuss some general approaches to achieve availability.

Approaches to achieve availability **Redundancy:** One way to meet availability is by replicating key components and data across numerous servers and data centers.
By doing this, we ensure that if one server fails or traffic is high, the load balancer can automatically reroute requests to an alternate backup server. Additionally , implementing redundant components across multiple layers (servers, databases, and networks) can prevent a single point of failure.

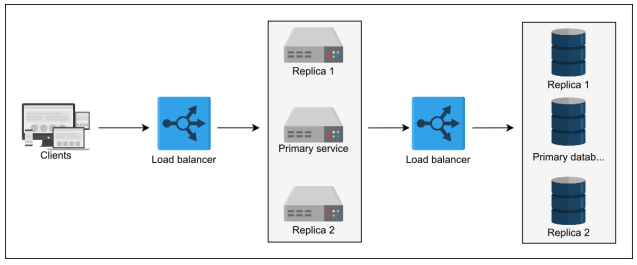


Figure 1.16: Replicating key components to eliminate single point of failure

Fault tolerance: During a discount sale on a shopping website, one of the key database nodes in a specific region suffers a hardware breakdown. This node handles a considerable amount of the user's activities in this

region. In such scenarios, our system must be fault-tolerant, meaning it will continue to function even if one or more components fail.
We can achieve this tolerance by using redundant components and failover methods that automatically switch traffic from the failed component to the backup component.
Rate limiting: Another approach to achieve availability is rate limiting.
The rate limiter restricts the number of requests that a service can handle. Setting rate limits to control the number of requests a user can make, thereby preventing system overload. For example, on a social media platform, a system overload would occur when users like posts, play videos, and follow others at a higher rate than usual.
Without rate limiting, this sudden increase in activity can overwhelm the system, leading to system failure.

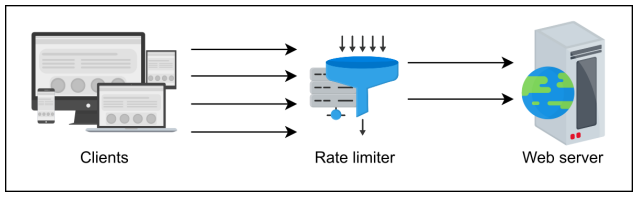
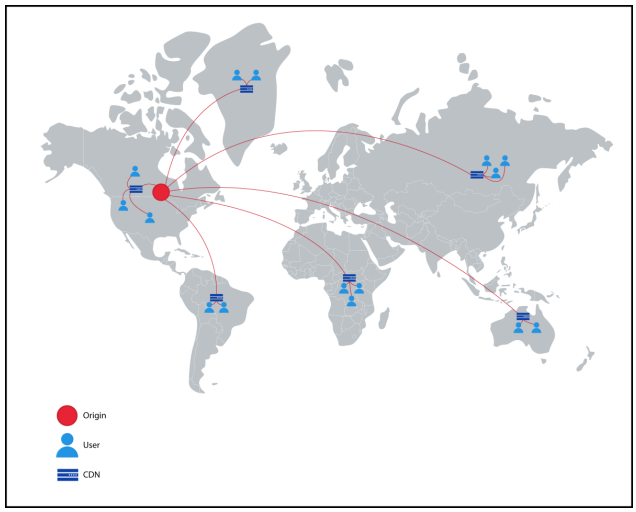


Figure 1.17: Rate limiting to prevent web server overload

CDNs: CDNs are cache servers distributed across different regions, which not only improve performance but also increase system availability by reducing the load on origin servers. Deploying servers in multiple geographical locations to ensure that regional outages do not affect overall system availability.
Also, they reduce latency for users in different locations.



Stress testing and monitoring: Another way to ensure availability is stress testing.
It is performed to determine how the system behaves under peak load conditions, allowing us to identify breaking points and ensure the system can handle sudden traffic spikes. This will prepare the system for availability after testing is complete.
Additionally, implementing monitoring can enable us to track system performance and detect anomalies in real-time.

3) Scalability

System scalability refers to a system's ability to expand and handle an increasing number of users while maintaining optimal performance.
For example, an interviewer might ask questions about designing a service like YouTube that can accommodate millions of users uploading and watching videos simultaneously, or designing a URL shortening service capable of handling billions of queries every day.

To address these questions about scalability in interviews, let's look at different approaches.

Approaches to achieve scalability **Manual scaling** is one approach to scaling applications. It involves either upgrading hardware on existing machines (vertical expansion) or adding more machines (horizontal expansion):

- **Vertical scaling (hardware upgrades):** Add more resources (RAM, CPU, storage) to existing machines for smaller demands. It's easier to manage since we aren't adding to the total number of machines.
- **Horizontal scaling (adding machines):** Increase the number of machines to distribute the workload for larger demands. In general, horizontal scaling is considered the preferred option for large-scale applications because it does not have a single point of failure and supports load balancing, unlike vertical scaling.

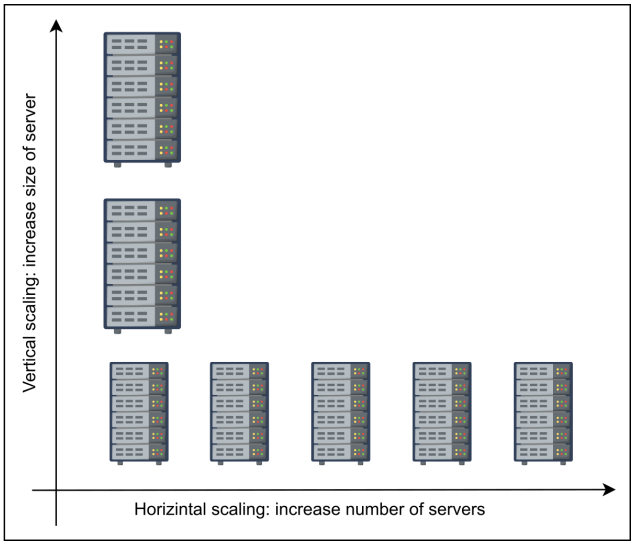


Figure 1.18: Vertical vs. horizontal scaling

Automatic scaling: Dynamically adjust resources (storage, processing power) based on demand to handle traffic spikes. This can be achieved using a cloud computing technique called Auto Scaling.
Sharding: Another approach to achieve scalability by dividing the database into shards to distribute the load across multiple servers. In this way, we distribute the data load between multiple servers. Key-rangeIt distributes data based on specific ranges of keys. and hash-basedThis distributes the data by applying a hash function to the keys, ensuring even distribution across shards. sharding are common techniques to perform sharding for databases.
Modular design: Break down the system into smaller, independent components, allowing each service to scale independently according to demand without affecting the performance of other services.

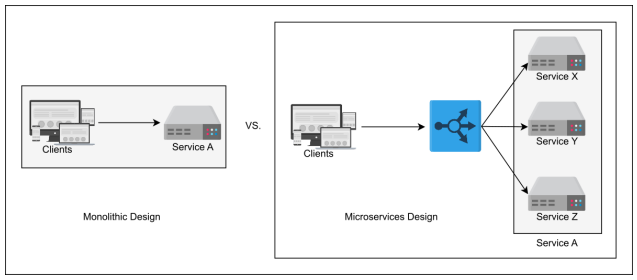


Figure 1.19: Monolithic vs. modular designs

Caches and CDNs: Caches store frequently accessed data in memory to reduce response times and database loads.
CDNs, on the other hand, are used to distribute static content to users rather than retrieving it from the origin, thereby further reducing the server's load. By utilizing caching and a CDN, the system can efficiently manage a high volume of user requests without experiencing slower performance.
Now that we've understood different approaches to achieving NFRs, including performance, availability, and scalability, let's practice them by taking a deeper look at the system design of Google Maps and YouTube.

6.2 Acing NFRs: Google Maps and YouTube

Let's explore non-functional requirements for Google Maps and YouTube System Design problems.

Design Google Maps

Designing a navigation system like Google Maps involves enabling users to identify their current location, find the optimal route based on specified destinations, and provide detailed turn-by-turn directions for seamless navigation.

Considering the following non-functional requirements of Google Maps, let's describe the strategies to achieve them:

- **High availability:** The design of Google Maps includes a large road network graph. If we hosted this graph on a server, it would definitely crash due to its large size and high user demands. To ensure availability, we divide the graph into smaller segments and host them on separate servers. By replicating these servers, we eliminate single points of failure and utilize a load balancer to distribute incoming user requests across multiple segment servers.

- **Scalability:** To scale our Google Maps system, we utilize a distributed architecture where each segment is hosted on a separate server, allowing us to serve user requests for different routes from different segment servers. Thus, we can serve millions of user requests. As we are using a modular design, we can easily add more segments to handle additional data.

Let's summarize the strategies we used to achieve Google Maps NFRS:

Nonfunctional requirements	Strategies
Availability	• Divide the road network graph into small graphs (segments) to process user queries. • Replicate the small segment servers. • Request load balancing across different segment servers.
Scalability	• Partition the large graphs into smaller graphs to ease segment addition. • Host the graphs on different servers to handle increased number of queries per second.

Design YouTube

Designing a video streaming platform like YouTube involves enabling users to stream videos, upload videos, search for videos by their titles, and like/dislike videos.

Considering the following non-functional requirements of YouTube, let's describe the strategies to achieve them:

- **Minimal response times:** To ensure the optimal performance of YouTube's design, we utilize multiple caching servers at both the ISP and CDN levels to deliver the most viewed content with the fastest response times. At the same time, choosing an appropriate storage system for different types of data, such as using Bigtable to store thumbnails and Blob storage to store videos, can reduce latency. We prefer to use a Lighttpd-based web server for users to upload their videos, as it processes such content more efficiently and provides a smoother user experience.

- **Reliability:** To ensure the system's high reliability, we utilize data sharding, which prevents any single type of data from affecting the others if it becomes unavailable. We replicate critical components to achieve system fault tolerance and eliminate faulty servers by monitoring their health using heartbeat messagesA node in a distributed system sends a regularly spaced message indicating it's healthy and active. If the node fails to send heartbeat messages, other nodes can assume that the node has failed..

Nonfunctional Requirements	Strategies
Less response time	• Cache at different layers • CDNs • Choose appropriate storage systems (e.g., blob storage to store videos, Bigtable to store thumbnails) • Serve videos and static content with Lighttpd
Reliability	• Data sharding • Replicate critical components • Heartbeat protocol

6.3 Quick tips for NFR interview questions

- Proactively ask questions to clarify non-functional requirements during the interview. For example:
  - Expected user traffic
  - Expected data load
  - Expected downtime tolerance
- Evaluate the trade-offs between different techniques, considering factors such as system complexity, cost, and maintainability.
- Prepare a list of commonly asked questions with their solutions. For example:
  - For reliable transaction processing—choose ACID-compliant relational databases.
  - For large-scale data applications, consider using NoSQL databases, such as MongoDB or Cassandra, to achieve scalability.
  - Real-time data processing and analytics—choose platforms like Apache Kafka, Amazon Kinesis, etc.

Remember, there is no one-size-fits-all solution. Every design decision involves trade-offs. As a designer of scalable systems, our ability to weigh these trade-offs is critical. It 's crucial to ask the interviewer clarifying questions, consider the NFRs carefully, and make informed choices to create a robust System Design.

6.4 Conclusion

In this lesson, we have attempted to demonstrate the importance of non-functional requirements and how to address them in System Design interviews. By understanding common NFRs and practical strategies for solving them, we will be better prepared to address NFR-related questions during the interview.

7 Let AI Evaluate Your Understanding of Non-Functional Requirements

We have now covered the non-functional attributes of a system. Take the following AI-powered assessments to solidify and test your knowledge of non-functional attributes in real-world scenarios. Don 't worry; our AI mentor will guide you along the way.

7.1 Assessment 1: Cloud-based file store

You are leading a team developing a cloud-based file-sharing platform for a multinational corporation. You 're tasked with ensuring that the platform has the following attributes:

- Depending on the time and need, the platform is able to handle an increasing number of requests arriving from different parts of the world.
- The users of the platform have worldwide 24/7 access to its files.

Keeping these requirements in view, state two of the most important and relevant nonfunctional requirements from the list provided below, such that their inclusion in the design will enable us to achieve the scenario mentioned above.

- Maintainability
- Scalability

- Fault tolerance
- Reliability
- Availability

Question:
State the two most important non-functional requirements for the file-sharing system
Reference Answer:
Scalability and Availability

7.2 Assessment 2: Financial trading platform

Imagine a banking application for financial transactions and buying online products. This platform allows users to obtain their account status, transfer money, pay utility bills, and generate bank statements.
List the following **non-functional requirements in the correct order,** starting from the most important non-functional requirement to the least important non-functional requirement:

- Low latency
- Consistency
- Security

Question:

Listing non-functional requirements in correct order
Reference Answer:

1. Security
2. Consistency
3. Low Latency

7.3 Assessment 3: Spacecraft operations

A space agency relies on critical systems to operate spacecraft, conduct space missions, and gather valuable data for scientific research. Imagine a scenario where a spacecraft is on a mission to explore a distant planet, and it encounters a hardware malfunction or a communication disruption with the control center on Earth. State **one of the most important and relevant non-functional requirements** from the list provided below, such that its inclusion in the design would enable us to recover from the scenario mentioned above. Please also provide proper **reasoning** behind your decision:

- Reliability
- Maintainability
- Consistency
- Fault tolerance
- Availability
- Scalability

Note: You must provide reasoning along with the name of the non-functional requirement in your answer.
Question:
State the most important non-functional requirement for this system along with proper reasoning
Reference Answer:
Fault tolerance {and reasoning}

7.4 Assessment 4: Twitter

Consider a scenario for Twitter's recently introduced feature allowing users to stream live video directly within their tweets. As a result, the number of users streaming live video has surged exponentially. Users are consequently experiencing delays, and the system struggles to handle the increased load.
State **one of the most important and relevant non-functional requirements** from the list provided below, such that its inclusion in the design would enable us to handle the scenario mentioned above.

- Reliability
- Maintainability
- Consistency
- Fault tolerance
- Availability
- Scalability

Question:
State the most important non-functional requirement for this system
Reference Answer:
Horizontal Scalability Note: The correct answer should have only 1 non-functional requirement.